Requirements-level reference specification of the LRE controller

8 September 2026

Source. The specification is derived from the requirements file `lre_requirement_all.txt` alone, which has 51 LRE entries and declares four operating modes. No repository content, implementation artefact or earlier draft was consulted. The interpretive decisions it needed are listed in Table 4.

Scope. The relation below specifies the mode changes, the handling of operator input, and the output events that go with them, over one sensor snapshot per step. It does not define the vehicle's dynamics, a CPA algorithm or an evasive manoeuvre. The statements labelled A01–A17 are interpretive decisions, not claims about what the requirements say.

State and observations. The modes are

`Mode = {OCM, MOM, HCM, CAM}`.

At power-up the mode is OCM; there is no separate initialisation mode. Two optional actuator records, `lastVel` and `lastHdng`, hold the most recently emitted values and are unspecified before the first output; the mode relation does not depend on them. [LRE-DM1, DM8, FR1, Beh1; A09]

A step observes one input

`e ∈ {none, reqVel(v), reqHdng(h), reqOCM, reqMOM, reqHCM, endTask}`,

where `v` and `h` are reals. `none` is a step with no operator input, not an interface event. A step whose input has no handler in the current mode can still take an autonomous transition. [LRE-ARCH2, DM6; A01, A03]

All derived quantities and guards in a step use the same sensor snapshot. Table 1 gives the symbols used below.

Table 1: Symbols used in the guards. The four thresholds are parameters with default 1; the distances are evaluated for the closest static obstacle (`cstc`) and the closest dynamic obstacle (`cdyn`) of the current snapshot.

Symbol	Meaning
`s`	`minSafeDist`
`H`	`staticObsHorizDist`
`V`	`staticObsVertDist`
`D`	`staticObsDfltVertDist`
`ds`	`odist(cstc)`
`dd`	`odist(cdyn)`
`hs`	`hdist(cstc)`
`vs`	`vdist(cstc)`
`P`	`inOpez`
`C`	`cda < s ∧ tcpa ≥ 0`

The four thresholds default to 1 and remain distinct parameters; a literal `1` in the transition table is a literal, not one of them. [LRE-DM4; A12]

For an obstacle `i` that exists, at the current depth,

`hdist(i) = sqrt(ns_rel_dist(i)² + ew_rel_dist(i)²)`

`vdist(i) = abs(depth − obs_depth(i))`

`odist(i) = sqrt(hdist(i)² + vdist(i)²)`.

An obstacle is static exactly when both horizontal velocity components are zero; its rate of climb plays no part. `cstc` and `cdyn` are the static and the dynamic obstacle at minimum **overall** distance; ties go to the smallest identifier, and `−1` stands for an empty class, so both indices range over `ℕ ∪ {−1}`. [LRE-DM2, DM3, DM5, SF1–SF3, OP3, OP4, Var5, Var6; A05, A06]

Before any guard is evaluated,

`hvel = sqrt(ns_vel² + ew_vel²)`

`vvel = rate_of_climb`

`vel = sqrt(hvel² + vvel²)`

`P = (ds ≤ s) ∨ (depth ≤ 0)`.

`CalcCStc` runs before `CheckOPEZ`, and `CalcCDyn` before `CalcCPA`. Everything used in a step comes from that step's snapshot: advice emitted during the step neither replaces the measured velocity nor causes the guards to be re-evaluated. [LRE-OP1–OP5, Var1–Var8, GP1; A01]

When a class of obstacle is absent, let `F` be a fixed real with

`F > max(0, 1, s, H, V, D)`.

At index `−1`, `odist = hdist = vdist = F`, and the four obstacle field accessors of DM5 are zero; these sentinel distances do not follow the geometry above. When `cdyn = −1`, `cda = F` and `tcpa = 0`. So the absence of dynamic obstacles never enables `C` and always enables `cda ≥ s`; the absence of static obstacles makes the obstacle part of `P` false but leaves the `depth ≤ 0` part. [LRE-DM5, OP2, OP5; A07]

When a dynamic obstacle exists, `cda ≥ 0` and `tcpa ∈ ℝ` are the results of `CalcCPA` for that step and are taken as observations. OP5 gives no formula relating them to position and velocity, so none is imposed: the mode relation is defined for every admissible observation, but it does not define a function from raw sensor values to commands. [LRE-OP5, Var7, Var8; A08]

How to read the relation. Table 2 gives the relation. An input trigger requires exactly that input. An `autonomous` trigger requires no operator event and is eligible whatever `e` is. Guards are evaluated in the source mode on the current snapshot.

For a step, let `E(m,e,x)` be the set of rows T02–T18 whose source is `m`, whose input trigger (if any) matches `e`, and whose guard holds in snapshot `x`. If the set is non-empty, exactly one of its rows is taken: the mode changes to that row's destination and exactly its output sequence is emitted. If the set is empty, the mode is kept and `[]` is emitted. The input is consumed either way. There is no priority among enabled rows, and no stuttering while a row is enabled. [A01–A04]

That is,

`R(m,e,x) = {(destination(t), outputs(t)) : t ∈ E(m,e,x)}` when `E(m,e,x) ≠ ∅`;

`R(m,e,x) = {(m, [])}` otherwise.

Each output updates its actuator record; the other record keeps its value. `[]` means no output event, not a zero command or a reset. Entry advice is already in the table, so it is not emitted a second time. Startup is the separate rule T01. [LRE-DM7, DM8, FR2, FR3; A09, A10]

Table 2: The transition relation. One row per behavioural requirement; an `autonomous` trigger needs no operator input. Where several rows are enabled at once, any one of them may be taken (A02).

Row / source requirement	Source	Trigger	Guard	Destination	Output sequence
T01 / LRE-Beh1	Power-up, outside the operating-mode relation	Power-up	`true`	OCM	`[]`
T02 / LRE-Beh2	OCM	`reqVel(v)`	`true`	OCM	`[advVel(v)]`
T03 / LRE-Beh3	OCM	`reqHdng(h)`	`true`	OCM	`[advHdng(h)]`

Table 2: The transition relation. One row per behavioural requirement; an `autonomous` trigger needs no operator input. Where several rows are enabled at once, any one of them may be taken (A02).

Row / source requirement	Source	Trigger	Guard	Destination	Output sequence
T04 / LRE-Beh4	OCM	`reqMOM`	`vel ≤ 1 ∧ ¬P ∧ dd > 1 ∧ ds > 1`	MOM	`[advVel(1)]` on entry
T05 / LRE-Beh5	MOM	autonomous	`P`	OCM	`[]`
T06 / LRE-Beh6	MOM	`reqOCM`	`true`	OCM	`[]`
T07 / LRE-Beh7	MOM	`endTask`	`true`	OCM	`[advVel(0)]`
T08 / LRE-Beh8	MOM	autonomous	`cda < s ∧ tcpa ≥ 0`	CAM	`[]` under A10
T09 / LRE-Beh9	MOM	autonomous	`hvel ≥ 1 ∧ hs ≤ H`	HCM	`[advVel(0)]` on entry
T10 / LRE-Beh10	MOM	autonomous	`vs ≤ D`	HCM	`[advVel(0)]` on entry
T11 / LRE-Beh11	MOM	autonomous	`vvel ≥ 1 ∧ vs ≤ V`	HCM	`[advVel(0)]` on entry
T12 / LRE-Beh12	MOM	`reqHCM`	`true`	HCM	`[advVel(0)]` on entry
T13 / LRE-Beh13	HCM	autonomous	`hs > H ∧ vs > V`	MOM	`[advVel(1)]` on entry
T14 / LRE-Beh14	HCM	autonomous	`cda < s ∧ tcpa ≥ 0`	CAM	`[]` under A10
T15 / LRE-Beh15	HCM	`reqOCM`	`true`	OCM	`[]`
T16 / LRE-Beh16	HCM	autonomous	`P`	OCM	`[]`
T17 / LRE-Beh17	CAM	`reqOCM`	`true`	OCM	`[]`
T18 / LRE-Beh18	CAM	autonomous	`cda ≥ s`	OCM	`[advVel(0)]`

The three autonomous MOM-to-HCM conditions are alternatives, not a conjunction. T10 has no velocity condition. T13 uses `V`, not `D`, and has no OPEZ or CPA veto; either would be a new guard. T18 has no `tcpa` condition. There is no autonomous exit from OCM, no transition from CAM to MOM or HCM, and no exit from CAM on OPEZ alone. [LRE-Beh4, Beh9–Beh14, Beh17, Beh18; A11]

Input handling where no row applies. Table 3 covers input-specific handling only; an autonomous row can still be taken in the same step.

Table 3: Handling of operator inputs by mode. An input with no explicit handler is consumed without effect; an autonomous row may still be taken in the same step.

Current mode	Inputs with explicit handlers	Inputs consumed without an input-specific action
OCM	`reqVel`, `reqHdng`, guarded `reqMOM`	`reqOCM`, `reqHCM`, `endTask`; also `reqMOM` when T04's guard is false
MOM	`reqOCM`, `reqHCM`, `endTask`	`reqVel`, `reqHdng`, `reqMOM`
HCM	`reqOCM`	`reqVel`, `reqHdng`, `reqMOM`, `reqHCM`, `endTask`
CAM	`reqOCM`	`reqVel`, `reqHdng`, `reqMOM`, `reqHCM`, `endTask`

In OCM, velocity and heading payloads are passed on unchanged, with no range check, clipping, normalisation or unit conversion. In the other modes such requests are neither forwarded nor kept for later. A request that arrives in the step that enters OCM is handled by the source mode and is not replayed after entry. [LRE-Beh2, Beh3, Beh19; A03, A14]

Ambiguities. The decisions in Table 4 complete the relation. When the specification is used as an oracle, their effects should be kept apart from what the requirements state directly.

Table 4: The list of ambiguities: each point the requirements leave open, and the reading used in this specification.

ID	Underdetermined point or tension	Resolution
A01	ARCH2 says guards are evaluated each step, but gives no timing model, event/sensor order, or rule for cascading transitions.	Use an atomic step with one coherent current sensor snapshot, fresh derived quantities, and at most one selected transition. Evaluate input-triggered and autonomous guards together against the source mode. Autonomous conditions are level-triggered, not rising-edge-triggered. Entry outputs do not cause another transition within that step.
A02	Several transitions can be enabled together; the file assigns no priority, including between operator commands, OPEZ, collision avoidance, and caution.	Admit every enabled row as a possible alternative, selecting exactly one. Requirement numbering and table order confer no priority. No extra veto is inferred. This is an explicitly nondeterministic resolution; it does not require a fixed safety or operator priority.
A03	There is no event-queue, retry, or blocked-request policy. Beh19 explicitly excludes some input handling but does not give a full rule for all modes.	Consume every presented input in its current step. Unhandled inputs and failed guarded requests have no input-specific effect. If an autonomous row is selected instead of an enabled input row, the input is still consumed. No buffering, deferral, or replay occurs.
A04	The file does not specify simultaneous operator events or whether enabled autonomous transitions may be postponed indefinitely.	Present at most one operator event per step. Simultaneous arrivals can be serialized in any order, with one step per event; the source fixes no order or intervening sensor evolution. With no operator event, steps still occur. Take some enabled row on each step; stutter only if no row is enabled. No fairness or real-time bound beyond this per-step rule is imposed.
A05	Var5 and Var6 call the indices natural numbers while also prescribing `-1`; ties between closest obstacles are not resolved.	Extend each index domain to `ℕ ∪ {-1}`. Among equally close obstacles of the appropriate class, use the smallest natural identifier. The tie rule can affect `hs`, `vs`, or CPA and is an interpretive choice.
A06	DM3 permits a partial function over natural identifiers without stating finiteness; minimum selection can fail for certain infinite collections.	Assume every operational obstacle register is finite. Static classification uses exactly DM2's two horizontal zero tests, even if an obstacle has nonzero vertical velocity.
A07	DM5 prescribes zero relative-position/velocity accessors for missing obstacles, while OP5 refers to large-distance defaults without defining the resulting CPA pair. "Safe large" has no numeric value.	Use the real sentinel `F` defined above for missing distances and `cda`, and choose `tcpa = 0` for no dynamic obstacle. Treat the missing-obstacle case explicitly; do not feed its zero relative-position accessors into an invented CPA formula. This is a chosen safe completion, not a formula stated in OP5.
A08	OP5 does not specify relative-velocity sign, two- versus three-dimensional CPA, treatment of a past CPA, or zero relative speed.	Leave CPA for existing dynamic obstacles abstract at the mode interface: provide a nonnegative real `cda` and real `tcpa` each step. Do not clamp negative `tcpa`, assume a collision cone, or add a numerical formula. A raw-sensor-level checker must separately identify its CPA convention; this reference does not independently validate that convention.
A09	Startup actuator values, OCM entry outputs, exit actions, and repeated in-mode advice are unspecified.	Emit nothing on startup or OCM entry unless a selected row explicitly supplies output. Emit MOM/HCM advice once on each actual entry. Empty-output steps preserve actuator records. No periodic reissue or exit output is added; initial record values remain unspecified.

Table 4: The list of ambiguities: each point the requirements leave open, and the reading used in this specification.

ID	Underdetermined point or tension	Resolution
A10	FR4 requires evasive manoeuvres in CAM but specifies no command, direction, speed, timing, or algorithm. Other general mode descriptions also do not specify ongoing output schedules.	For this mode-transition reference, use a minimal output completion: emit only the outputs in the table, with no additional CAM entry or in-mode output. This does not specify or discharge FR4's physical manoeuvre obligation. A comparison of evasive-command algorithms cannot be supported by this file alone.
A11	Descriptive text such as "potential collision" or "reduced velocity" could invite extra guards, broader exits, or inferred actions absent from the numbered behaviours.	Use the numbered guards literally. In particular, HCM entry advises exactly zero; `endTask` acts only in MOM; negative `tcpa` alone does not release CAM; and OPEZ alone does not release CAM. No additional transitions or safety vetoes are inferred from mode names.
A12	DM4 gives defaults but no valid configuration range or relationship between thresholds. Several guards use literal 1, and some use different static thresholds.	Keep thresholds as finite real parameters with default 1 and no added ordering or positivity precondition. Keep all literal 1 values literal. Require `F` to exceed every relevant threshold. No hysteresis or equivalence between `V` and `D` is inferred.
A13	Equality at thresholds and the sign of vertical velocity are common implementation choices, but the relevant prose here is explicit.	Use exact real comparisons with the strictness shown in the table; add no tolerance. Use signed `vvel = rate_of_climb`, not its absolute value. This records that no unresolved equality decision was needed for these guards.
A14	Payload validity, stale data, sensor failures, and numeric exceptional values are not specified.	Model finite real sensor values and request payloads and coherent snapshots. Forward OCM payloads exactly. NaN, infinities, hardware failures, malformed events, and stale-snapshot recovery are outside this reference's input domain, not newly invented controller behaviours.
A15	DC1 requires one transition per Beh1–Beh18 and forbids duplicate source/trigger combinations, yet several behaviours are autonomous exits from the same mode.	Keep exactly one labelled row per Beh1–Beh18, including initialization and the two pass-through transitions. Read source/event uniqueness as applying to explicitly named input events; retain distinct autonomous requirement rows with their own guards. DC1 supplies no priority or global guard-disjointness rule. If it was intended to forbid all multiple autonomous exits, it conflicts with the listed behaviours.
A16	No sampling frequency, environmental evolution, actuator-to-plant delay, or liveness guarantee is given. Overlapping entry and exit conditions can cause repeated mode changes across steps.	State only the per-step relation. Do not promise eventual stability, eventual OCM, or physical collision avoidance. Permit repeated transitions allowed by unchanged or changing observations; do not insert dwell times, debounce, or stabilization logic.
A17	Different enabled rows can have the same destination but different output obligations. It is unspecified whether their actions should be merged.	Execute only the selected row's actions plus its destination-entry advice, as already combined in the table. Do not merge enabled rows' actions. Preserve row identity when recording a witness; for conformance compare both destination and output sequence.

Boundary and overlap cases. Each line of Table 5 concerns the named guard only; another enabled row may still decide the outcome.

Table 5: Boundary and overlap cases, read from the strictness of the comparisons in Table 2.

Observation	Consequence
`vel = 1`	Satisfies T04's velocity conjunct.
`dd = 1` or `ds = 1`	Fails T04's corresponding strict-distance conjunct.

Table 5: Boundary and overlap cases, read from the strictness of the comparisons in Table 2.

Observation	Consequence
`ds = s` or `depth = 0`	Makes `P` true.
`cda = s`	Disables T08/T14 and enables T18, regardless of `tcpa`.
`cda < s` and `tcpa = 0`	Enables the collision guard `C`.
`cda < s` and `tcpa < 0`	Disables `C`; also does not enable T18. CAM therefore remains CAM unless `reqOCM` is selected.
`hvel = 1` and `hs = H`	Enables T09.
`vs = D`	Enables T10 at any velocity.
`vvel = 1` and `vs = V`	Enables T11.
`vvel = −1` and `vs ≤ V`	Does not satisfy T11's velocity condition; T10 may independently apply.
`hs = H` or `vs = V`	Fails T13. Both distances must strictly exceed their respective thresholds.

For example, in CAM with input `reqOCM` and `cda ≥ s`, both T17 and T18 are eligible, so `(OCM, [])` and `(OCM, [advVel(0)])` are both allowed: the move to OCM does not by itself fix whether velocity advice was emitted.

In MOM, if `P` and `C` hold and at least one static-caution guard holds, OCM, CAM and HCM are all permitted destinations; HCM comes with `[advVel(0)]`, the other two with nothing. This overlap is intended (A02).

With no obstacles, positive depth and no operator input: HCM goes to MOM, CAM goes to OCM with `[advVel(0)]`, MOM stays in MOM, and OCM stays in OCM until a guarded `reqMOM` arrives. If depth is not positive, `P` holds even with no obstacles, so HCM can have both T13 and T16 enabled.

Nothing guarantees stability, even under constant observations. When `D > V`, an observation with `hs > H` and `V < vs ≤ D` enables HCM-to-MOM by T13 and then MOM-to-HCM by T10 on the next step; with no competing guard the modes alternate and the entry advice alternates between 1 and 0. The requirements permit this, and this specification does not repair it.

Use as an oracle. A single-step observation conforms exactly when its `(destination, output sequence)` is in `R(m,e,x)`, starting from OCM. An implementation with a fixed priority selects a subset of the allowed alternatives and need not exercise them all; a mode change or output outside the relation is a violation under the decisions recorded here. A trace conforms when every step does; no fairness condition is added.

Differences of priority alone are therefore not detected. Findings that turn on event consumption, obstacle ties, the missing-obstacle CPA or unspecified outputs depend on the readings in Table 4 and should be reported as such. The CPA arithmetic for existing obstacles and the effectiveness of evasive manoeuvres are outside this relation's scope. No extracted model, mutant, verifier result or kill rate informed the relation.